

QA Automation Technical Assessment - Solution Explanation

Profile Level: Software Test Engineer - approximately 3+ years experience

Automation Stack: Playwright + JavaScript + Page Object Model + JSON test data

1. Assignment Understanding

The assignment requires automation of an end-to-end Employee Lifecycle Management flow on the OrangeHRM demo application. The expected coverage is valid login, employee creation with a profile image, search and edit of job information, API-based validation, deletion verification, logout, a maintainable POM framework, HTML reporting, video recording, and clear setup/run documentation.

2. Chosen Approach

I selected Playwright with JavaScript because it provides a simple test runner, strong auto-waiting, UI and API support in the same framework, built-in HTML reporting, screenshots, traces, and video recording. The solution is intentionally practical rather than over-engineered so that it represents the work of a QA engineer with around 3+ years of automation exposure.

Component	Purpose
tests/employeeLifecycle.spec.js	Single readable end-to-end business workflow using test.step blocks and assertions.
pages/LoginPage.js	Login, Dashboard verification support, and logout actions.
pages/PimPage.js	PIM navigation, employee create/search/delete actions.
pages/EmployeeDetailsPage.js	Job tab handling and update of Job Title / Employment Status.
pages/ApiClient.js	Public REST API simulator calls for create/update/delete validation.
data/employee.json	Data-driven employee test data.
utils/testData.js	Loads JSON and generates a unique Employee ID at runtime.
playwright.config.js	Browser, timeout, HTML report, video, screenshot, and trace settings.

3. Automated Flow

- 1. Login:** Open OrangeHRM login page, enter Admin/admin123, and verify Dashboard is visible.
- 2. Add Employee:** Navigate to PIM > Add Employee. Read employee data from JSON, generate a unique Employee ID, upload profile.png, save, and confirm the Personal Details page.
- 3. API Create Check:** Send the same employee values to a public API simulator and assert returned name and Employee ID against UI input data.
- 4. Search and Edit:** Search the created employee using Employee ID, open the employee, go to Job, update Job Title and Employment Status, and verify the success toast and selected values.
- 5. API Update Check:** Send the updated employee data through the API helper and assert that API response values match the data updated from the UI flow.
- 6. Delete:** Delete the employee in OrangeHRM, search again, and confirm the employee row is absent. Also validate a successful API DELETE response.
- 7. Logout / Session:** Logout, verify the login page, then try a protected Dashboard URL and confirm the session redirects to login.

4. API Validation Note

The assessment allows either the OrangeHRM API or a public test API. The public OrangeHRM demo does not provide a stable assessment-specific API contract, so this solution uses JSONPlaceholder as an API simulator. This demonstrates request creation, response validation, and UI/API data consistency without claiming that JSONPlaceholder persists OrangeHRM employees. Its write operations are simulated, so deletion is validated using the successful DELETE response.

5. Reporting and Evidence

Playwright HTML reporting is enabled. Video is configured for every run, while screenshots and traces are retained on failure. After local execution, the real HTML report appears under playwright-report/ and runtime media appears under test-results/. The package contains a clearly labelled pre-run HTML execution template only; it does not fabricate pass/fail results.

6. How to Run

Step	Command
Install dependencies	npm install
Install browser	npx playwright install chromium
Create environment file	Copy .env.example to .env
Run test	npm test
Run headed	npm run test:headed
Open HTML report	npm run report

7. Practical QA Considerations

- Employee ID is generated dynamically to reduce conflicts because the OrangeHRM demo is shared by many users.
- Assertions include meaningful messages to make failures easier to understand.
- Page objects contain reusable UI actions so locators are not duplicated in the test.
- Job Title and Employment Status are kept in the data file so master-data changes can be adjusted without changing the test flow.
- The framework uses one Chromium project and one end-to-end spec to keep the solution appropriate for the requested experience level.

8. Files Included in Submission

The ZIP contains source code, Playwright configuration, dependency definition, JSON test data, profile image, README, functional coverage, and a sample pre-run execution report. This PDF provides the brief implementation explanation. Before final GitHub submission, the candidate should run the suite locally and commit/share the generated Playwright HTML report and a selected test-run video if the reviewer expects runtime evidence inside the repository.

Prepared as a concise technical explanation suitable for a Software Test Engineer with approximately 3+ years of experience.